305141
Introduction to Internet of Things

Dr. Settha Tangkawanit
Department of Electrical and Computer Engineering
Faculty of Engineering
Naresuan University

Course Outine

Part 1: Introduction to Internet of Things

- Overview and Meaning
- Internet of Things Component
- Internet of Things Architecture

305141
Introduction to Internet of Things

Chapter 02
IoT System Architectures

Dr. Settha Tangkawanit
Department of Electrical and Computer Engineering
Faculty of Engineering, Naresuan University

Typical Components

- We will study typical components used for networks, databases, etc.
- Figure 2.1 shows the organization of an IoT system:

- The ***plant*** or *environment* is the physical system with which the IoT system interacts. We will use these two terms interchangeably.

- A set of ***devices*** form the leaves of the network. A node may include sensors and/ or actuators, processors, and memory. Each node has a network interface. A node may or may not run the Internet Protocol.

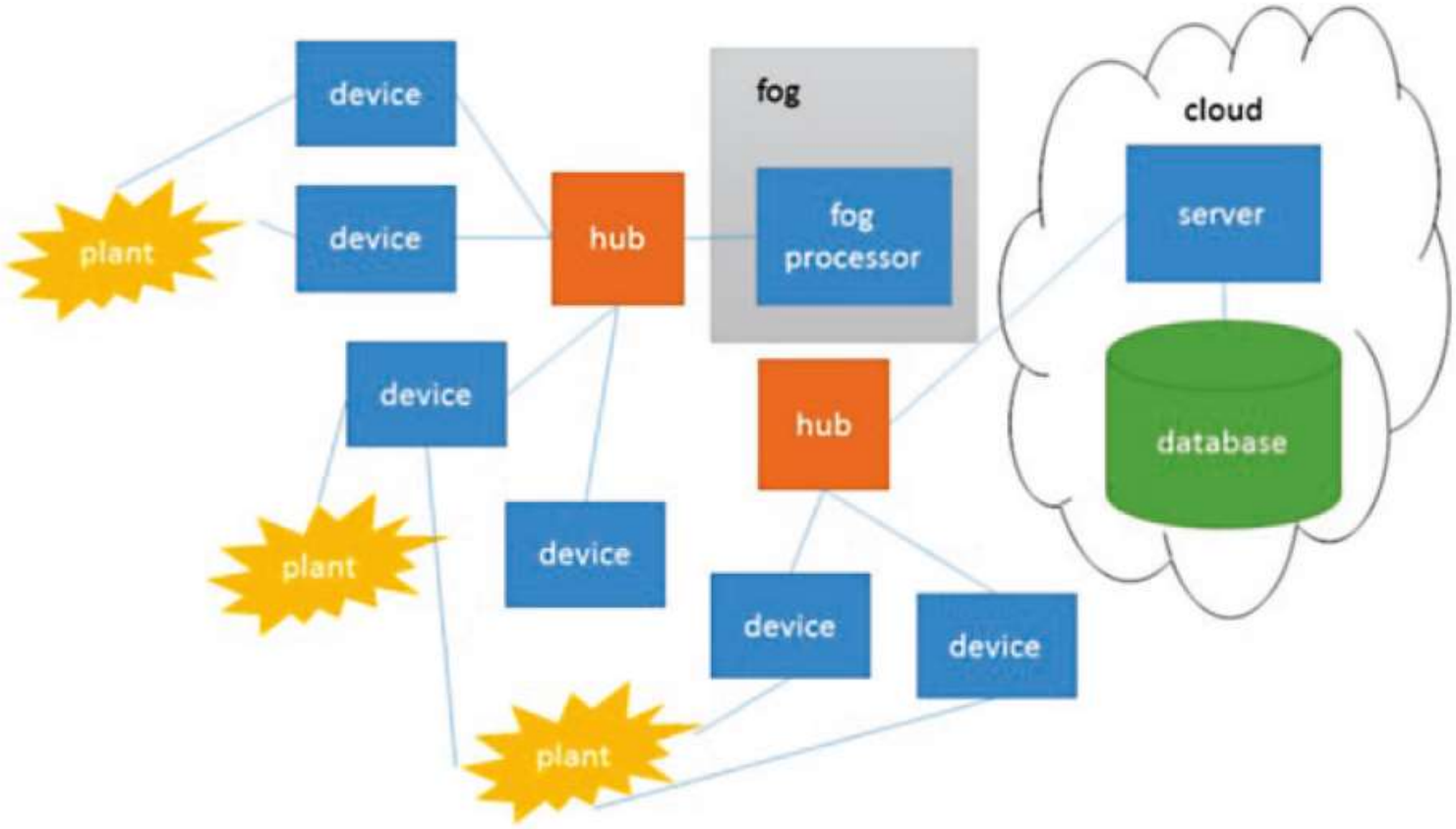
Typical Components

- ***Hubs*** provide first-level connectivity between the nodes and the rest of the network. Hubs are typically run IP.

- ***Fog processors*** perform operations on local sets of nodes and hubs. Keeping some servers nearer the nodes reduces latency. However, fog devices may not have as much compute power as cloud servers. Fog devices also introduce system management issues.

- ***Cloud servers*** provide computational services for the IoT system. ***Databases*** store data and computational results. The cloud may provide a variety of services that mediate between nodes and users.

IoT System Architectures



Protocols Concepts

- Several protocols are used for data services in IoT systems.
- Communication protocols may not provide sufficient abstraction for many applications.
- IoT systems need multi-hop, end-to-end communication. They also may exhibit complex relationships between data sources and sinks.
- Higher-level protocols can provide services that model more closely the needs of IoT systems

Protocols Concepts

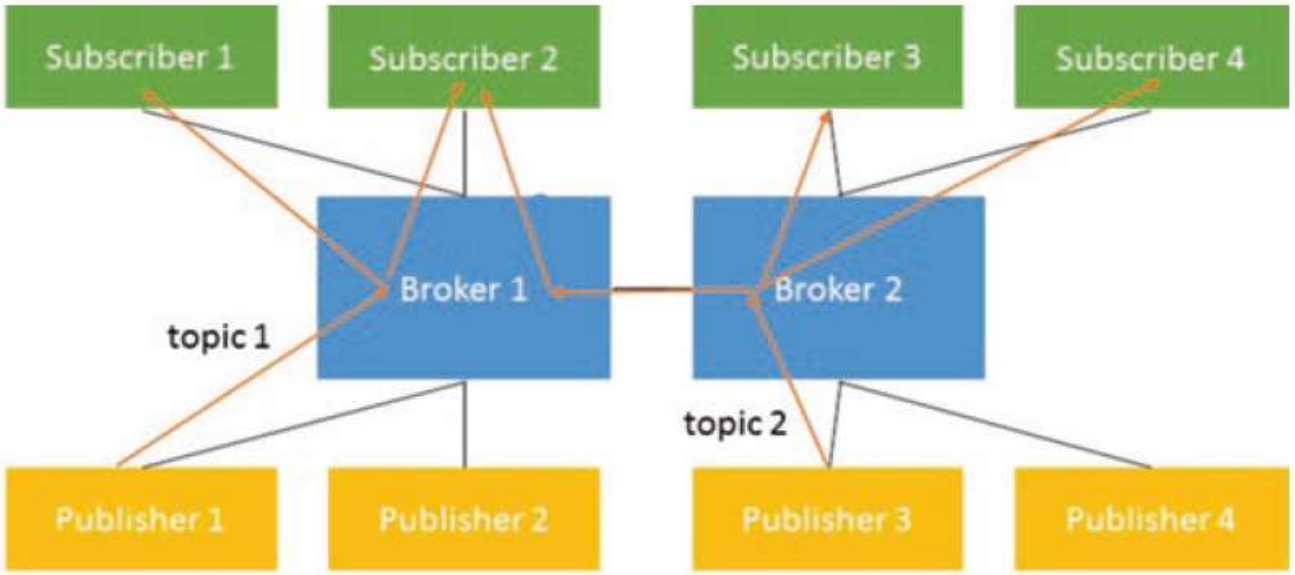
- Given the heterogeneous and long-lived nature of most IoT systems, standards are often used rather than custom protocols.
- Several different protocols have been proposed and, to varying degrees, used for IoT systems [Duf13].
- The user space has not yet converged on a single standard for IoT communication services.
- Given the prevalence of event-oriented models in IoT systems, a protocol should support event-style communication.

Protocols Concepts

- The **HTTP** protocol uses a request/response design pattern. A client issues a request for a hypertext object; the server then replies with the object in response.
- A ***publish/subscribe*** protocol [Twi11] requires less coupling between the client and server as illustrated in Fig. 2.2. The server, known as a publisher, classifies messages into categories. Clients subscribe to the categories of interest to the
- Publish/subscribe systems are typically mediated by *brokers* which receive published messages from publishers and send them to subscribers. Messages may be organized by topic; all message of a given *topic* are distributed by the brokers to the subscribers for that topic.

Protocols Concepts

- The broker knows the identities of subscribers but the publisher does not. Brokers may interact with each other using a bridge protocol.
- A *bridge* allows indirect publication of messages, with a message going from the publisher to a first broker, then to a second broker, and finally to subscribers who are not connected to the first broker.



Protocols Concepts

- Data Distribution Service (DDS) (http://portals.omg.org/dds/) [Obj16] is a publish/ subscribe software architecture; several implementations of DDS are in use.
- A DDS *domain* maintains a logical *global data space*; the data is managed over a set of local stores.
- Publishers and subscribers are dynamically discovered across the network.
- Publishers can specify a number of quality of service parameters that are enforced by the brokers.

Protocols Concepts

- Real-Time Publish/Subscribe Protocol (RTPS) [Obj14] is a so-called wire protocol that defines a protocol for communication with DDS and other publish/subscribe systems.
- RTPS provides QoS properties, fault tolerance, and type safety.

- Esposito et al. [Esp09] developed an architecture for time-sensitive publish/subscribe systems that would be scalable to Internet-sized systems.
- They identified three major design goals: predictable latency, guaranteed delivery in the presence of multiple faults, and continued performance under scaling.

Protocols Concepts

- Their architecture has three abstraction layers: the network layer consists of domains composed of nodes; the nodes layer consists of clusters, with each cluster's members belonging to the same stub domain; and a coordinators layer.
- The coordination layer routes messages using a tree-based topology built on top of a distributed hash table.
- The coordinator is *p*-redundant to provide fault-tolerant coordination.
- To provide fault-tolerant overlays, they formulate a model for path diversity that can be computed with limited knowledge of the network connections.

Protocols Concepts

- Kang et al. [Kan12] used a semantics-aware communication mechanism to reduce overhead and improve reliability.
- They use state-space estimators at both the publisher and subscriber to maintain continuity of sensor values in the presence of network variations.
- Their state estimator is of the form $xk + 1 = Fk + 1xk$.
- The designer sets a model precision bound δ for each sensor.
- The bound is used to manage bandwidth requirements.
- Their system also dynamically adjusts the model precision bound.

Protocols Concepts

- Choi et al. [Choi16] combined DDS with the OpenFlow software-defined networking protocol to ensure that DDS can implement the QoS parameters.
- They added two QoS parameters that could not be easily deduced from the standard DDS parameters: MINIMUM_SEPARATION and an E2E_LATENCY specified by subscribers.

IoT-Oriented Protocols

- We can divide protocols into two major categories: those that are tied to a specific physical layer and those that are not.
- Generally speaking, protocols that rely on a specific physical layer do not use the Internet Protocol, while protocols that are physical layer agnostic do use IP.

IoT-Oriented Protocols: Zigbee

- **Zigbee** [Zig14, Far08] is a mesh network designed for low-power operation.
- A variety of derivative application standards specialize the protocol for applications such as smart homes and utilities.
- Zigbee is based on the IEEE 802.15.4 PHY and MAC standards.
- 802.15.4 operates in three bands: 868 MHz, 915 MHz, and 2.4 GHz.
- It delivers bit rates from 20 to 250 kbps, depending on the frequency band.
- The Zigbee NWK layer sits on top of the 802.15.4 MAC layer and provides data and management services.
- The APL layer includes three sections: the application support sublayer, the Zigbee Device Objects layer, and the application framework.

IoT-Oriented Protocols: Zigbee

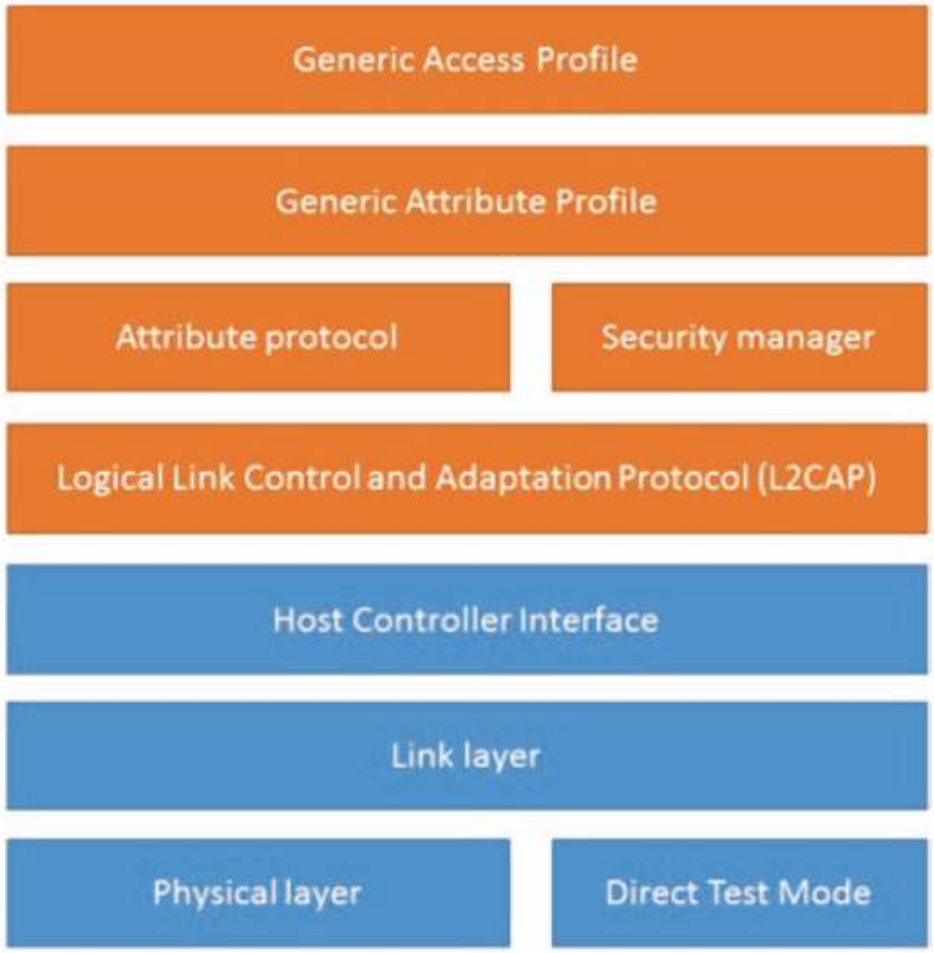




IoT-Oriented Protocols: Bluetooth Low Energy

- Bluetooth Low Energy (BLE) (https://www.bluetooth.com/what-is-bluetoothtechnology/ how-it-works/low-energy) [Hay13] is a part of the Bluetooth standard designed for low-power operation such as devices powered from coin cell batteries.
- A BLE device can work as a transmitter, receiver, or both. Figure 2.3 illustrates the *Bluetooth Classic* protocol stack.
- The link layer provides an advertising service; devices can scan to identify nodes and networks.
- Devices can act as gateways to the Internet based on network address translation.
- The BLE protocol is stateful. BLE includes a number of optimizations to reduce power consumption.

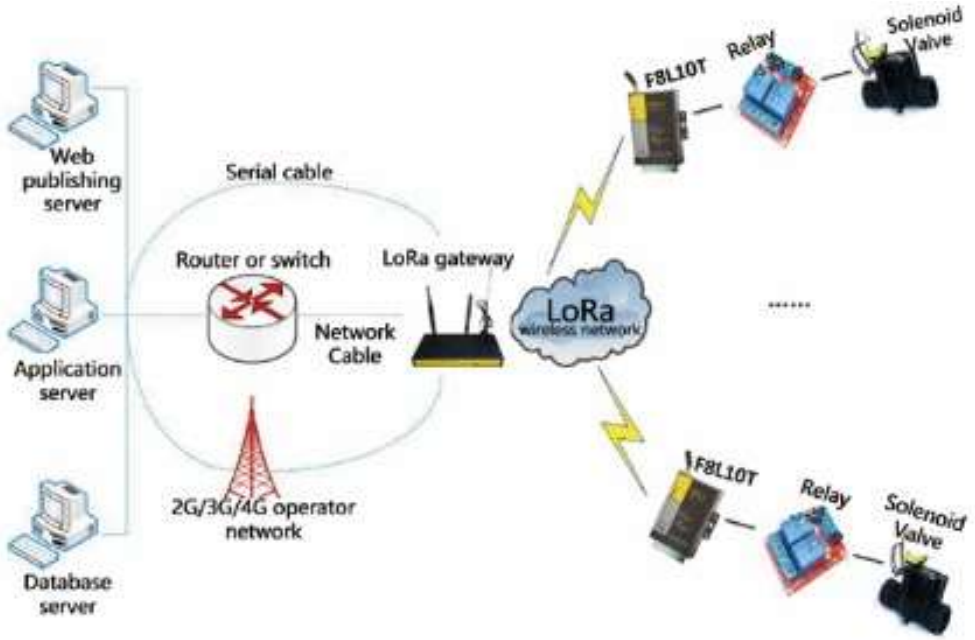
IoT-Oriented Protocols: Bluetooth

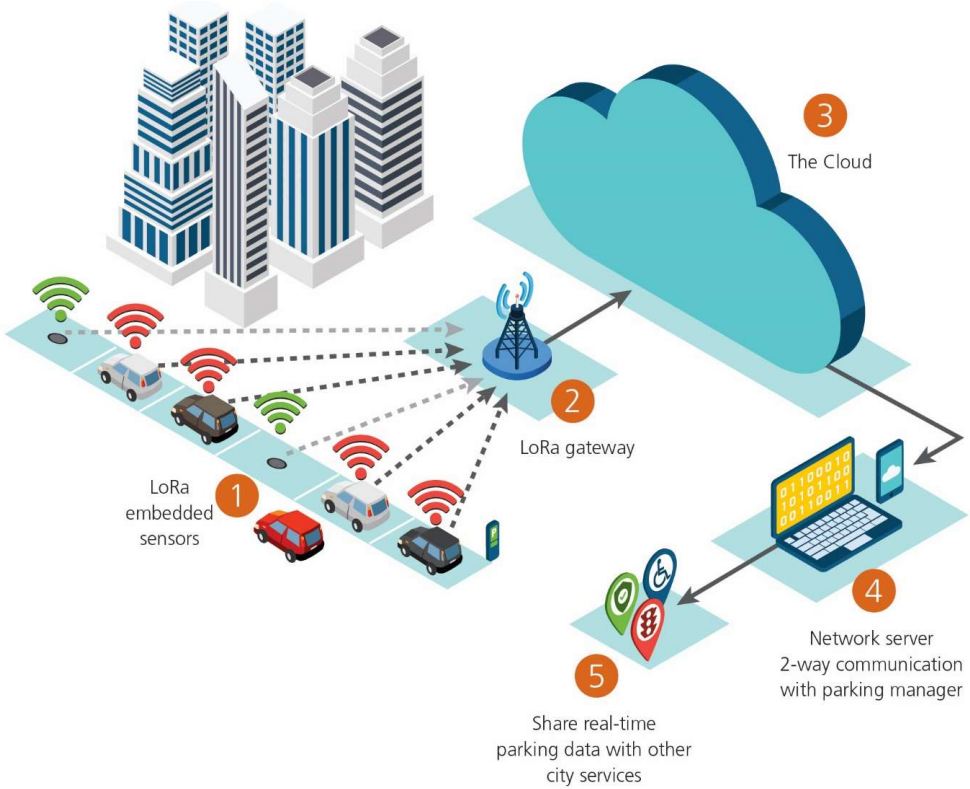


IoT-Oriented Protocols: LoRa

- LoRa (http://lora-alliance.org) [LoR15] is designed for wide-area IoT applications with a base station covering hundreds of square kilometers.
- It is designed to support a network topology with gateways for end devices, with gateways organized into their own star network.
- Data rates range from 0.3 to 50 kbps.

IoT-Oriented Protocols: LoRa





IoT-Oriented Protocols: MQTT

- MQTT (http://www.mqtt.org) [IBM12, Oas14] is an IoT-oriented protocol with publish/subscribe semantics.
- The protocol is designed for low overhead and is agnostic to the data payload.
- MQTT provides three levels of quality of service: *at most once* provides best-effort service, *at least once* assures delivery but may incur duplicates, and *exactly once* ensures the message is delivered without duplication.
- MQTT is based on a publish/subscribe model. A message is given a retention attribute when it is published; messages with QoS designations of *at least once* or *exactly once* should set the retention flag.
- A new subscriber to the topic will receive the last publication on that topic.

IoT-Oriented Protocols: XMPP, REST

- XMPP (http://xmpp.org) is a protocol for streaming XML. It provides security, authentication, and information about network availability, and rosters of clients.
- XMPP-IoT (http://xmpp-iot.org) is a dialect of XMPP designed for IoT applications.

- REST [Vaq14, Rod15] is widely used for Web services and has received some use as an IoT service model. REST is a design pattern for stateless HTTP transfers.
- It exposes directory-structured form resource indicators. REST can be used to transfer XML or JSON data. Clients access resources using GET, PUT, POST, and DELETE methods.

IoT-Oriented Protocols: CoAP, GCM

- CoAP (http://coap.technology) [IET14] is a REST-based Web transfer protocol designed for IoT devices. It can be used with several types of data payloads, including XML and JSON.

- Google Cloud Pub/Sub [Goo17A, Goo17B] can be used to provide publish/subscribe service to IoT and other systems. Topics and subscriptions are exposed as REST collections.
- The system is divided into a data plane for messages and a control plane for allocation to servers known as *routers*; data plane servers are known as *forwarders*.
- The routers balance consistency and uniformity of data using a consistent hashing algorithm.

IoT-Oriented Protocols: AWS IoT, MS Azure

- Amazon Web Services (**AWS**) IoT [Bar15] is a managed cloud service for IoT devices, which are termed *things*.
- A *thing shadow* is a cloud model of a thing. A rule engine transforms messages based on rules and routes the results to AWS services.
- The *message broker* is based on MQTT. A *Thing Registry* assigns unique identity to things.

- Microsoft Azure (https://azure.microsoft.com/en-us/services/iot-hub/) provides IoT-oriented services. Its Service Fabric is a middleware communication system that supports microservices running on a cluster. A microservice may be either stateless or stateful.

Databases

- Databases are used for both short-term and long-term storage.
- Applications may rely on databases to retrieve data over a time window for analysis.
- Some use cases may require archival storage of values.
- Unstructured databases, known as *noSQL*, are used in many IoT systems.
- A noSQL database does not have a schema. Simple noSQL databases represent data as key-value pairs, but other representations are possible.
- The lack of a schema allows quick deployment but may cause maintenance problems.

Time Bases

- Many IoT systems require a notion of global time.
- Several algorithms, starting with Lamport's algorithm [Lam78], have been developed for the synchronization of clocks in a distributed system.
- The Network Time Protocol (RFC1305) is used on the Internet for distributed time synchronization.

Security

- Security is a system property; the system can be only as secure as its weakest component.
- Security features are provided by components at several layers in the IoT stack: devices, physical networks, and middleware.
- A unified view of IoT system security architectures has not yet emerged.
- Some, but not all processors for low-power operation, provide security features such as encryption accelerators and root of trust.

Security

- The National Security Agency has developed families of lightweight block ciphers [Sch13]: SIMON targets hardware implementations, and SPECK is intended for software implementations.
- Gulcan et al. [Gul14] developed a low-power implementation of SIMON.
- Several networks provide security features. Bluetooth Low Energy provides a Simple Secure Pairing protocol to protect against passive eavesdropping.
- It also provides address randomization. As discussed above, Zigbee provides two network security models: centralized and distributed.
- LoRa provides unique network keys, unique application keys, and device-specific keys.